System Programming — Homework 2

Multi-Process File Search: `procSearch`

Dicle ÇOBAN
220104004088

March 27, 2026

Contents

1 Introduction **2**

2 Design Decisions **2**
2.1 How I Partitioned Directories 2
2.2 How Workers Communicate with the Parent 2
2.3 How I Structured the Signal Handlers 3
2.4 How I Avoided Async-Signal-Safety Issues 3
2.5 Why I Used sigsuspend Instead of pause 3
2.6 How I Fixed the SIGCHLD Race Condition 4
2.7 Pattern Matching 4

3 Test Scenario Screenshots **5**
3.1 Test Setup 5
3.2 Test 1: Basic Pattern Search 5
3.3 Test 2: Pattern Search with Minimum File Size 6
3.4 Test 3: No Matching Files 6
3.5 Test 4: SIGINT Handling (Ctrl+C) 6

4 Conclusion **7**
4.1 Challenges I Faced 7
4.2 Final Result 7

1 Introduction

For this homework, I implemented a command-line program called `procSearch`. It searches for files matching a given pattern inside a directory tree. To speed up the search, I used multiple worker processes that run at the same time.

The user runs the program with four arguments: a root directory (`-d`), a number of worker processes (`-n`), a filename pattern (`-f`), and an optional minimum file size (`-s`).

My overall design is simple: the parent process reads the top-level subdirectories of the root, divides them among the workers, and then waits. Each worker searches its own directories and, when done, sends a signal to the parent. After all workers finish, the parent collects the results and prints them as a directory tree with a summary at the end.

I kept the design as simple as possible. Workers do not share memory with each other or with the parent. All communication goes through signals and temporary files.

2 Design Decisions

2.1 How I Partitioned Directories

I only look at the first level of subdirectories under the root. I distribute them to workers using a **round-robin** approach:

```
    for (int i = 0; i < num_subdirs; i++) {
        int w = i % num_workers;
        wdirs[w][wdir_count[w]++] = subdirs[i];
    }
```

For example, with 7 subdirectories and 4 workers:

- Worker 0 gets directories 0 and 4
- Worker 1 gets directories 1 and 5
- Worker 2 gets directories 2 and 6
- Worker 3 gets directory 3

If there are fewer subdirectories than workers, I reduce the worker count automatically. If there are no subdirectories at all, I skip forking and let the parent search the root directly.

2.2 How Workers Communicate with the Parent

I use two mechanisms together:

1. **Exit code**: Each worker calls `exit(match_count % 256)` when it finishes. The parent reads this value with `waitpid()` to find out how many matches that worker found.

2. **Temporary files**: Before forking, I create one temporary file per worker using `mkstemp()`. Each worker writes its matching file paths into this file. After all workers are done, the parent reads these files to build the full list of matches.

I register the temporary files with `atexit()` so they are automatically deleted when the parent exits.

2.3 How I Structured the Signal Handlers

I handle four signals in this program:

- **SIGUSR1** (worker → parent): A worker sends this when it finishes searching. The handler just increments a counter (`g_sigusr1_count`). When the count equals the number of workers, the parent knows all workers are done.
- **SIGCHLD** (OS → parent): The OS sends this when a child process exits. My handler calls `waitpid(-1, &status, WNOHANG)` in a loop to immediately reap all finished children. I save their PID and exit status in arrays for later use.
- **SIGINT** (Ctrl+C → parent): The handler sets a flag (`g_sigint_received = 1`). The main loop checks this flag and calls `terminate_all_workers()`, which sends SIGTERM to all children and waits up to 3 seconds. If they do not exit in time, it sends SIGKILL.
- **SIGTERM** (parent → worker): The worker handler sets a flag (`g_sigterm_received = 1`). The worker checks this flag between directory entries and exits cleanly when it is set.

2.4 How I Avoided Async-Signal-Safety Issues

POSIX says that signal handlers must only call async-signal-safe functions. Functions like `printf`, `fprintf`, and `malloc` are not safe to call inside a handler.

I made sure none of my handlers call unsafe functions:

- `handler_sigusr1`: only increments a counter.
- `handler_sigint`: only sets a flag to 1.
- `handler_sigchld`: only calls `waitpid()` (which is async-signal-safe) and writes to integer arrays.
- `worker_sigterm_handler`: only sets a flag to 1.

All output and processing happens in the main loop, not inside handlers. I also declared all handler flags as `volatile sig_atomic_t` so the compiler does not optimize them away.

2.5 Why I Used sigsuspend Instead of pause

The parent waits for workers by looping until `g_sigusr1_count` reaches the expected number. I originally used `pause()` to sleep between checks, but there is a race condition: a SIGUSR1 signal can arrive right after the loop checks the condition but before `pause()` is called. In that case, the parent misses the signal and waits forever.

I fixed this by using `sigsuspend()` instead:

```c
    while (g_sigusr1_count < (sig_atomic_t)g_num_workers) {

        if (g_sigint_received) {
            fprintf(stderr, "\n[Parent] SIGINT received. Terminating all worke
            terminate_all_workers();
            fprintf(stderr, "[Parent] Partial shutdown complete.\n");
            fprintf(stderr, "Matches found so far: %d\n", num_entries);
            for (int i = 0; i < num_subdirs; i++) { free(subdirs[i]); }
            return 1;
        }

        if (g_sigchld_received) {
            g_sigchld_received = 0;
            process_sigchld(worker_results, &num_results);
        }

        sigsuspend(&mask_waiting);
    }
```

`sigsuspend()` atomically sets the signal mask and suspends the process. This removes the race condition completely.

2.6 How I Fixed the SIGCHLD Race Condition

I found a subtle race at the end of the program. A worker can send SIGUSR1 and immediately call `exit()`. This triggers SIGCHLD, and my handler reaps the worker right away. Then, when the main code tries to call `waitpid(pid, &status, 0)` for that worker, the child is already gone. `waitpid()` returns `-1` with `ECHILD`, and that worker's match count is lost from the summary.

I fixed this by blocking SIGCHLD during the result-collection phase:

```c
    sigset_t block_chld, old_sigmask;
    sigemptyset(&block_chld);
    sigaddset(&block_chld, SIGCHLD);
    sigprocmask(SIG_BLOCK, &block_chld, &old_sigmask);
```

This way, I always check the reaped-PID buffer first before calling `waitpid()`, so no worker's result is ever lost.

2.7 Pattern Matching

I implemented a custom pattern matcher in `pattern.c`. The only special operator is `+`, which matches one or more occurrences of the character immediately to its **left**. For example, `rep+ort` matches `report`, `repport`, and `reppport`.

My implementation uses recursion with backtracking. When I see that the next character in the pattern is `+`, I find the longest run of the repeat character in the input, then try to match the rest of the pattern by backtracking from the end of that run.

I also apply the pattern to the filename stem (the part before the first dot). This means `rep+ort` matches `report.txt` and `report.c` as well.

3 Test Scenario Screenshots

3.1 Test Setup

I created a test directory under `/tmp/crawltest` with the following structure:

```
/tmp/crawltest/
  dir1/  report.txt  report.c  data.txt  report.pdf
  dir2/  repport.txt  reppport.txt  image.png  large.bin
  dir3/  hello.c  world.h  notes.txt
  dir4/  report.txt  test.py  config.json
```

3.2 Test 1: Basic Pattern Search

Command:

```
./procSearch -d /tmp/crawltest -n 2 -f "rep+ort"
```

This should match all files named `report`, `repport`, or `reppport` (with any extension).

```
diclesaracoban@Dicle-MacBook-Pro HW2 % ./procSearch -d /tmp/crawltest -n 2 -f "rep+ort"
[Worker PID:19115] MATCH: /tmp/crawltest/dir2/reppport.txt (99 bytes)
[Worker PID:19115] MATCH: /tmp/crawltest/dir2/repport.txt (300 bytes)
[Worker PID:19116] MATCH: /tmp/crawltest/dir4/report.txt (20 bytes)
[Worker PID:19116] MATCH: /tmp/crawltest/dir1/report.c (19 bytes)
[Worker PID:19116] MATCH: /tmp/crawltest/dir1/report.txt (62 bytes)
[Worker PID:19116] MATCH: /tmp/crawltest/dir1/report.pdf (24 bytes)

/tmp/crawltest
|-- dir1
|------ report.c (19 bytes) [Worker 19116]
|------ report.pdf (24 bytes) [Worker 19116]
|------ report.txt (62 bytes) [Worker 19116]
|-- dir2
|------ repport.txt (300 bytes) [Worker 19115]
|------ reppport.txt (99 bytes) [Worker 19115]
|-- dir4
|------ report.txt (20 bytes) [Worker 19116]

--- Summary ---
Total workers used  : 2
Total files scanned : 14
Total matches found : 6
Worker PID 19115 : 2 matches
Worker PID 19116 : 4 matches
```

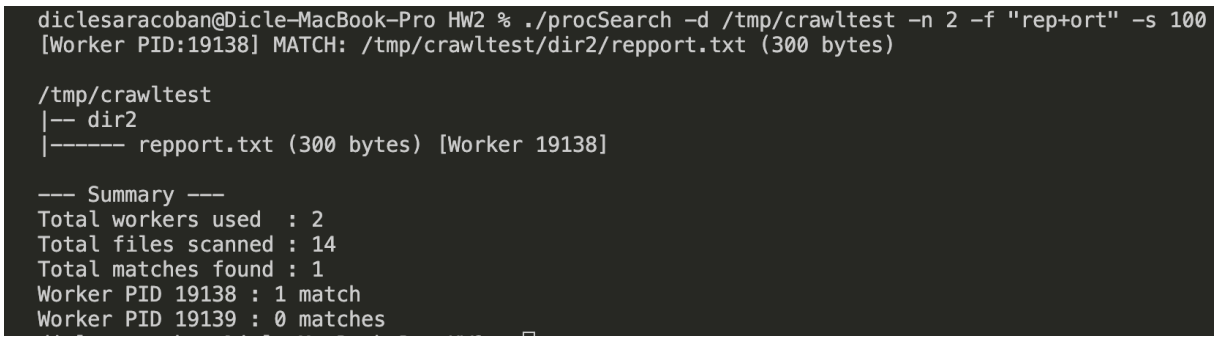
Figure 1: Basic pattern search with `rep+ort`

3.3 Test 2: Pattern Search with Minimum File Size

Command:

```
./procSearch -d /tmp/crawltest -n 2 -f "rep+ort" -s 100
```

Only files that match the pattern and are at least 100 bytes should appear.



```
diclesaracoban@Dicle-MacBook-Pro HW2 % ./procSearch -d /tmp/crawltest -n 2 -f "rep+ort" -s 100
[Worker PID:19138] MATCH: /tmp/crawltest/dir2/repport.txt (300 bytes)

/tmp/crawltest
|-- dir2
|------ repport.txt (300 bytes) [Worker 19138]

--- Summary ---
Total workers used  : 2
Total files scanned : 14
Total matches found : 1
Worker PID 19138 : 1 match
Worker PID 19139 : 0 matches
```

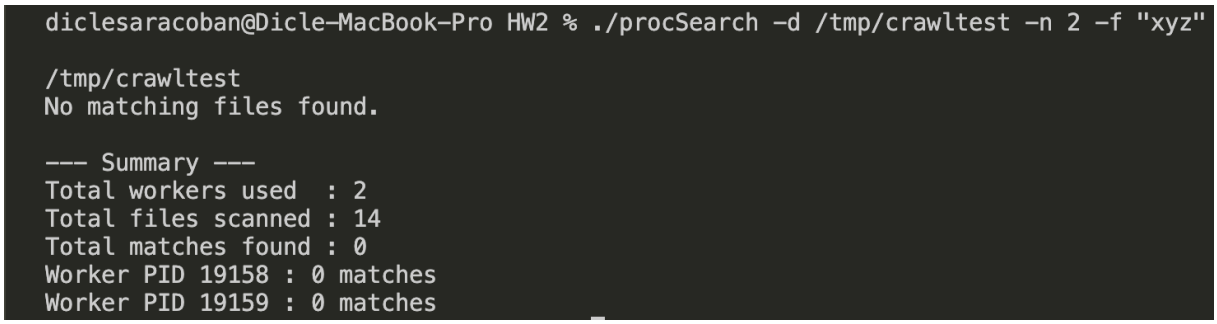
Figure 2: Pattern search with minimum file size filter

3.4 Test 3: No Matching Files

Command:

```
./procSearch -d /tmp/crawltest -n 2 -f "xyz"
```

No files match this pattern, so the output should say "No matching files found." and the summary should show 0 matches.



```
diclesaracoban@Dicle-MacBook-Pro HW2 % ./procSearch -d /tmp/crawltest -n 2 -f "xyz"

/tmp/crawltest
No matching files found.

--- Summary ---
Total workers used  : 2
Total files scanned : 14
Total matches found : 0
Worker PID 19158 : 0 matches
Worker PID 19159 : 0 matches
```

Figure 3: Search with no matching files

3.5 Test 4: SIGINT Handling (Ctrl+C)

I started the program on a large directory and pressed Ctrl+C while it was running. The parent received SIGINT, sent SIGTERM to all workers, waited for them to stop, and printed a partial shutdown message.

```
diclesaracoban@Dicle-MacBook-Pro HW2 % ./procSearch -d / -n 4 -f "rep+ort"
[Worker PID:19202] MATCH: //usr/share/man/mann/report.n (21827 bytes)
[Worker PID:19204] MATCH: //private/tmp/flattest/report.txt (6 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir2/reppport.txt (99 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir2/repport.txt (300 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir4/report.txt (20 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir1/report.c (19 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir1/report.txt (62 bytes)
[Worker PID:19204] MATCH: //private/tmp/crawltest/dir1/report.pdf (24 bytes)
[Worker PID:19204] MATCH: //opt/homebrew/Cellar/sbcl/2.5.6/share/sbcl/contrib/sb-sprof/report.lisp (11082
bytes)
^C
[Parent] SIGINT received. Terminating all workers...
[Worker PID:19203] SIGTERM received. Partial matches: 0. Exiting.
[Worker PID:19204] SIGTERM received. Partial matches: 8. Exiting.
[Worker PID:19202] SIGTERM received. Partial matches: 1. Exiting.
[Parent] Partial shutdown complete.
Matches found so far: 0
```

Figure 4: Graceful shutdown on SIGINT

4 Conclusion

4.1 Challenges I Faced

1. SIGCHLD Race Condition. The hardest problem I ran into was a race between the SIGCHLD handler and the final `waitpid()` call. When a worker finished very fast, the handler reaped it before my fallback `waitpid()` ran. That worker's match count disappeared from the summary. I fixed it by blocking SIGCHLD with `sigprocmask()` during the result-collection phase and always checking the reaped-PID buffer first.

2. sigsuspend vs pause. I first used `pause()` to wait for SIGUSR1. Then I realized that if the signal arrived just before `pause()`, the program would hang forever. Switching to `sigsuspend()` solved this because it atomically unblocks signals and suspends, with no gap in between.

3. Pattern Matching Semantics. My first implementation treated `+` as applying to the character on the right. After my instructor corrected me, I rewrote it to apply to the character on the **left**. This required changing the recursion logic and adding backtracking.

4. Async-Signal-Safety. I had to be careful never to call `printf` or similar functions inside signal handlers. I moved all output to the main loop and kept handlers as simple as possible.

4.2 Final Result

My program compiles with zero `-Wall -Wextra` warnings. It correctly:

- Forks 2–8 worker processes and distributes directories in round-robin order.
- Uses SIGUSR1 to notify the parent when a worker is done.
- Reaps all child processes immediately via the SIGCHLD handler, so no zombie processes are created.
- Handles SIGINT gracefully by shutting down all workers cleanly.
- Collects results from temporary files and displays them as a tree with a summary.